

AXIOM Architecture v1.0

Investment Research Operating System — Technical Design Blueprint

基線版本：AXIOM Research Engine v0.7.0

文件狀態：Architecture Baseline / Pre-v1.0

日期：2026-07-21

適用範圍：v0.8–v1.0 產品與工程演進

核心定位

Evidence → Knowledge → Research → Impact → Valuation → ETF / Portfolio

文件控制

欄位	內容
文件名稱	AXIOM Architecture v1.0 — Design Blueprint
架構基線	v0.7.0 verified repository
目標版本	v1.0.0
主要讀者	產品負責人、研究工程師、資料工程師、估值模型開發者、測試與發佈維護者
規範度	「必須」為 v1.0 gate ；「應」為預設設計 ；「可」為延伸能力
變更原則	重大架構決策以 ADR 記錄 ；模型 schema 變更需 migration 與 backward compatibility 評估

目錄

目錄將於 Microsoft Word 開時更新。

1. 執行摘要

AXIOM 已從 v0.4 的研究與估值基礎，演進至 v0.7 的因果衝擊傳導。v1.0 的目的不是再堆疊功能，而是將既有能力整理成一個可維護、可校準、可追溯、可擴充的投資研究作業系統。

v1.0 的核心架構將以八個領域邊界組成：Canonical、Evidence、Research、Industry、Valuation、ETF、Scenario/Impact、Portfolio。所有領域以穩定 ID、來源證據、版本化快照與可重現執行紀錄互通。

v1.0 必須解決三個根本問題：

- 可信度：任何研究結論、Driver 分數、Catalyst 機率與變動，都能回溯到來源與計算方法。
- 一致性：資料模型、因果方向、快照語義、CLI 輸出與 public artifacts 具有一致契約。
- 可演進性：Evidence、Scenario、Portfolio 等新能力能在不破壞既有 Research / ETF / Impact 邏輯下加入。

v1.0 成功定義

維度	v1.0 成功條件
研究可追溯性	Claim → Evidence → Source 可完整追溯，且研究 Driver / Catalyst 能列出 supporting 與 contradicting claims。
推理可重現性	相同資料版本與相同參數，必須產生相同 Snapshot ID 與輸出。
模型可校準性	財務影響係數不再硬編碼於 engine，而由 DriverImpact / EstimateSensitivity / ModelPolicy 提供。
ETF / Portfolio 可聚合性	公司、主題、情境風險可按權重聚合，明確揭露 coverage、missing data 與 confidence。
工程可靠性	ruff、pytest、schema validation、CLI smoke、migration、public build 全部通過 release gate。

2. 架構原則與非目標

2.1 架構原則

原則	設計含義
Evidence-first	研究結論不得只有文字與信心分數；必須能連接到可識別的 Claim 與 Evidence。
Cause → Effect	所有可傳播 Industry Edge 一律以原因指向結果；逆向查詢由 traversal 參數處理，不改變 edge 語義。

Snapshot over mutation	對外研究、估值、Impact、ETF 與 Portfolio 結果採不可變快照；修正以新 revision 或 supersedes 關係表達。
Deterministic core	核心計算不依賴非確定性副作用；LLM 僅負責 extraction / classification 候選，最終寫入須經 schema 與 policy 驗證。
Explicit uncertainty	所有推論都需要 confidence、coverage、missing inputs 與 methodology。
Ports and adapters	Domain 與 Engine 不直接依賴 JSON 檔案、Web API 或特定 LLM；透過 repository 與 provider interfaces 隔離。
Backward-compatible CLI	既有 validate / research / industry / value / etf / impact / build-public 指令在 v1.0 仍可用。
Synthetic data disclosure	示範 ETF 或研究籃子必須標示 synthetic / non-tradable，避免與真實產品混淆。

2.2 非目標

- v1.0 不以即時交易執行、券商下單或風控限額系統為目標。
- v1.0 不宣稱 Impact Engine 輸出為精確價格預測；輸出是可追溯的情境估算。
- v1.0 不要求一次支援所有市場資料供應商；先建立 provider contract。
- v1.0 不以分散式微服務為前提；採 modular monolith，保留未來服務化邊界。

3. v0.4-v0.7 現況盤點

3.1 能力演進

版本	主要能力	架構價值
v0.4	Research Graph、Valuation、public JSON、validation	建立究與的可執行骨架。
v0.5	Industry Graph、Exposure、Path query	建立產業因果與供應鏈關係。
v0.6	ETF Profile、Holding、Theme Exposure、ETF Valuation	將公司研究聚合至 ETF。
v0.7	Shock、PropagationRule、Company / ETF Impact Snapshot	建立 deterministic causal propagation。

3.2 v0.7 實體資料目錄

```

data/
  canonical/  entities, securities, sources, evidence, relations
  ingestion/  raw_articles, article_admissions, entity_mentions, extracted_claims
  research/   theses, drivers, catalysts, driver_impacts, revisions, snapshots
  industry/   edges, exposures, graph_snapshots
  valuation/  facts, estimates, assumptions, scenarios, profiles
  etf/        profiles, holdings, theme_exposures, valuation_snapshots
  impact/     shocks, propagation_rules, impact_scenarios
  generated/  executions, valuation_books, valuation_snapshots
  public/     entity, ETF and impact artifacts

```

3.3 v0.7 Python package

```

src/axiom_engine/
  cli.py
  config.py
  io.py
  repository.py
  models/{core,research,industry,valuation,etf,impact}.py
  services/{validator,research,industry,valuation,etf,impact,public_builder}.py

```

3.4 現況優點

- 模型與服務分離已成形，Pydantic 提供 schema boundary。
- CLI 為端到端驗收入口，且 public JSON build 已形成發佈介面。
- Snapshot 與 deterministic ID 的觀念已在 valuation / impact 使用。
- Industry Graph 已統一 cause-to-effect，能成為 Scenario 與 Portfolio risk 的基礎。

3.5 主要架構債務

問題	影響	v1.0 對策
io.py / repository.py 邊界偏薄	Domain service 可能直接知道檔案布局	定義 Repository Protocol 與 Unit of Work。
Evidence 與 ingestion 尚未形成完整 pipeline	Driver / Catalyst 仍偏靜態	建立 Evidence Engine 與 admission workflow。
財務影響係數硬編碼	難以校準、公司差異不足	引入 DriverImpact、EstimateSensitivity、ModelPolicy。
多路徑 impact 僅保留 strongest path	可能低估共同衝擊或重複計算	增加 path aggregation policy 與 dependency deduplication。
Public schema 未正式版本化	前端或下游容易被破壞	加入 schema_version、generated_at、compatibility policy。
缺少 Portfolio domain	無法描述持倉層級風險與情境	v0.9 建立 Portfolio Foundation。

4. v1.0 目標架構

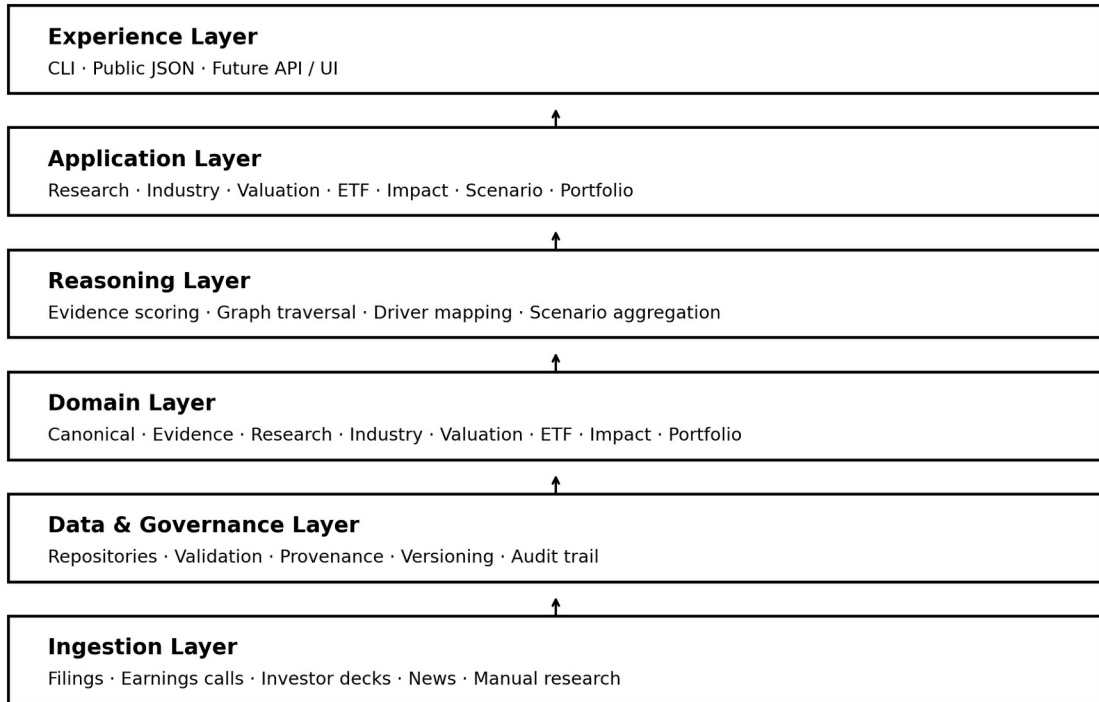


圖 1：AXIOM v1.0 邏輯分層。上層體驗只依賴 Application Contracts；底層資料來源透過 adapters 進入。

4.1 分層責任

層級	責任	不得包含
Experience	CLI、public JSON、未來 REST/GraphQL、研究 UI	Domain 計算與資料庫細節。
Application	Use case orchestration、transaction boundary、authorization policy	底層檔案格式與供應商 API。
Reasoning / Engine	Evidence scoring、Graph traversal、Scenario aggregation、Valuation computation	CLI formatting、I/O side effects。
Domain	Entity、Claim、Driver、Catalyst、Shock、Snapshot 等模型與 invariants	外部 SDK、網路呼叫。
Data & Governance	Repository、schema migration、	商業規則的隱性實作。

	provenance 、 audit 、 validation	
Ingestion	Source provider 、 parser 、 normalizer 、 dedupe 、 admission queue	直接修改已發布 Snapshot。

4.2 Modular Monolith 邊界

```

axiom_engine/
domain/
  canonical/ evidence/ research/ industry/ valuation/ etf/ scenario/ portfolio/
application/
  commands/ queries/ dto/ policies/
engines/
  evidence/ graph/ research/ valuation/ impact/ scenario/ portfolio/
ports/
  repositories.py providers.py clock.py id_factory.py
adapters/
  json_store/ sqlite/ llm/ market_data/ filings/ web/
presentation/
  cli/ public_builder/ api/
governance/
  validation/ migrations/ audit/ release/

```

v1.0 可先保留現有模組路徑，逐步遷移至上述結構；不要求一次性大改。每次遷移必須保持 CLI 與資料契約可用。

5. 領域模型藍圖

5.1 Canonical Domain

Canonical Domain 定義 AXIOM 中所有可被引用的穩定對象。任何其他 domain 只儲存 canonical ID，不複製實體名稱作為主鍵。

模型	關鍵欄位	v1.0 規則
Entity	entity_id, entity_type, canonical_name, aliases, status	ID 永久穩定；合以 redirect / supersedes 表達。
Security	security_id, issuer_id, exchange, ticker, currency	公司與證券分離；估值結果需明確指定 company/security。
Theme	theme_id, taxonomy_path, parent_id	支援階層 taxonomy 與版本。
Relation	relation_id, source_id, target_id, relation_type	非傳播關係與 IndustryEdge 分離。
Source	source_id, source_type,	保留合法可儲存 metadata；原文

	publisher, published_at, uri_hash	內容依授權策略管理。
--	-----------------------------------	------------

5.2 Evidence Domain

模型	用途	必要欄位
RawDocument	原始文件 metadata 與內容定位	document_id, source_id, content_hash, fetched_at, parser_version
DocumentChunk	可引用段落或頁面區段	chunk_id, document_id, locator, text_hash
Claim	標準化可驗證主張	claim_id, subject_id, predicate, object/value, valid_time, confidence
EvidenceLink	Claim 與 chunk 的支持/反駁關係	evidence_link_id, claim_id, chunk_id, stance, relevance, extraction_method
ExtractionRun	模型或規則抽取執行	run_id, provider, model, prompt_version, input_hash, output_hash
AdmissionDecision	是否納入 canonical evidence	decision_id, candidate_id, status, reviewer, reason

Evidence 寫入流程必須區分 candidate 與 admitted。LLM 產出一律先進 candidate store，通過 schema、dedupe、entity resolution 與 admission policy 後，才可影響 Driver / Catalyst。

5.3 Research Domain

模型	v1.0 擴充	說明
InvestmentThesis	status, horizon, conviction, evidence_summary	Thesis 是一組 Driver / Catalyst / Risk 的可發布觀點。
ResearchDriver	score, trend, momentum, materiality, confidence, updated_at	由靜態描述升級為 Dynamic Research Signal。
DriverObservation	as_of_date, score_delta, evidence_count, method	保留時間序列，避免直接覆寫。
Catalyst	expected_window, probability, status, dependencies	由單一 status 升級為 timeline event。
CatalystObservation	probability_delta, schedule_delta, evidence links	記錄每次更新原因。
ResearchRevision	author/agent, rationale, changed_fields, supersedes	支援人工與自動變更審計。

ResearchSnapshot	immutable published state	public build 與下游 engine 只讀 snapshot。
------------------	---------------------------	--------------------------------------

Driver Score 建議標準化為 0–100；Trend 為 rising / stable / falling；Momentum 為指定窗口的分數變化。分數方法必須由 ScorePolicy ID 指定，不能只存在程式註解。

5.4 Industry Domain

模型	關鍵規則
IndustryEdge	source_id → target_id 固定為 cause → effect；edge_type、strength、confidence、lead_lag_months、active range。
PropagationRule	elasticity、attenuation、mode、aggregation_policy、valid_from/to。
IndustryExposure	公司或 ETF 對 technology/theme/demand driver 的曝險權重與來源。
GraphSnapshot	一個可重現的 graph_version，列出使用的 edge/rule 版本。

Edge type taxonomy 建議分為 enables、drives_demand_for、constrains、consumes、supplies、substitutes_for。只有被標記 propagating=true 的 edge 參與 Impact Engine。

5.5 Valuation Domain

模型	v1.0 角色
FinancialFact	已實現財務事實，含 period、unit、source、restatement status。
Estimate	未來財務預測，含 scenario、analyst/model origin、confidence。
ValuationAssumption	multiple、discount rate、margin、shares、net debt 等輸入。
ValuationProfile	可重用模型配置與模型權重。
ValuationExecution	一次執行的 input refs、engine version、timestamp、status。
ValuationSnapshot	deterministic result；同樣輸入不得產生不同 snapshot。
ValuationBook	公司/證券/情境下的多模型集合與 blended fair value。

EstimateSensitivity	Driver 或 Shock 對 revenue/margin/EPS 等 estimate 的 elasticity。
---------------------	--

5.6 ETF Domain

模型	v1.0 規則
ETFProfile	fund identity、benchmark、strategy、currency、synthetic flag。
ETFHolding	as_of_date、weight、security/company mapping、source。
ETFThemeExposure	必須標示 derived/manual、methodology、coverage。
ETFValuationSnapshot	weighted upside、coverage、missing companies、contributors。
ETFImpactSnapshot	scenario/shock impact、coverage、confidence、contributors。

ETF 快照不得假設所有 holding 都有公司 valuation。Coverage 必須以總權重計算並在輸出中置頂。

5.7 Scenario / Impact Domain

模型	用途
Shock	單一外生變化：entity、direction、magnitude、unit、duration、confidence。
Scenario	多個 Shock、共同假設、相關性與時間軸。
ImpactNode	每個 entity 的 impact、confidence、lag、path attribution。
CompanyImpactSnapshot	財務與估值影響，來源 paths 與 sensitivity refs。
ETFImpactSnapshot	holding-weighted 聚合、coverage、contributors。
ScenarioComparison	Bull/Base/Bear 或自訂 scenario 的差異與排序。

v1.0 應新增 aggregation policy：strongest_path、independent_sum、correlated_sum、bottleneck。預設不得默默相加多條共享依賴路徑，以避免 double counting。

5.8 Portfolio Domain

模型	用途
----	----

Portfolio	portfolio_id 、 owner/scope 、 base currency 、 policy refs。
Position	security/ETF/cash 、 quantity 、 weight 、 cost basis 、 as_of_date。
LookThroughExposure	ETF 穿透後的 company/theme/driver 曝險。
PortfolioValuationSnapshot	weighted upside 、 coverage 、 currency normalization。
PortfolioImpactSnapshot	scenario downside/upside 、 contributors 、 concentration。
PortfolioRiskPolicy	單一公司、主題、國家、情境等限額與 warning。

6. Engine Contracts

所有 Engine 必須是純計算優先：輸入為已驗證 DTO / Snapshot refs，輸出為結果 DTO；資料讀寫由 Application service 控制。

Engine	輸入	輸出	關鍵不變量
EvidenceEngine	RawDocument / chunks / extraction policy	Claim candidates, EvidenceLinks	同一 input_hash + model/prompt version 可追蹤；不直接發布研究。
ResearchEngine	admitted claims, prior snapshot, score policy	Driver/Catalyst observations, draft revision	所有變更帶 evidence attribution。
IndustryGraphEngine	GraphSnapshot, source/target query	paths, exposure, propagation topology	只沿 active cause → effect edge。
ValuationEngine	facts, estimates, assumptions, profile	Execution, Snapshot, Book	deterministic IDs；模型失敗可局部降級。
ImpactEngine	Shock/Scenario, graph, sensitivities	node/company/ETF impacts	methodology 、 coverage 、 confidence 不可省略。
ScenarioEngine	scenario set, aggregation policy	comparison, rankings, deltas	多 shock 組合需顯式 correlation policy。
PortfolioEngine	positions, ETF holdings, snapshots	look-through exposure, valuation, impact	避免 ETF 與底層持股重複計算。

6.1 Impact Engine v1.0 計算鏈

```
Shock / Scenario
→ Graph propagation
→ DriverImpact / EstimateSensitivity
→ Revenue / Margin / EPS deltas
→ Valuation model re-run or sensitivity approximation
→ CompanyImpactSnapshot
→ ETF aggregation
→ Portfolio look-through aggregation
```

v0.7 的 revenue $\times 1.15 \times 1.10$ 係數可保留為 DemoModelPolicy，但 production profile 必須引用具名 sensitivity records。

6.2 Evidence Scoring 建議

```
claim_confidence = extraction_confidence  $\times$  source_quality  $\times$  evidence_relevance
```

```
driver_evidence_score = weighted_support - weighted_contradiction
```

```
driver_score_t = clamp(driver_score_t-1 + policy(driver_evidence_score, decay, materiality), 0, 100)
```

公式僅作契約示意；實際 policy 需版本化。任何 policy 更新不得回寫舊 Snapshot，而應產生新的 ResearchSnapshot。

7. 端到端工作流程

7.1 Evidence-to-Research

1. Provider 取得 filing、earnings call、investor deck、news 或人工輸入。
2. 建立 RawDocument，計算 content hash，執行 dedupe。
3. Parser 產生 DocumentChunk 與 locator。
4. Extraction adapter 產生 Claim candidates、Entity mentions 與 relation candidates。
5. Validator 執行 schema、entity resolution、temporal checks、duplicate detection。
6. Admission policy 將候選標記 admitted / rejected / needs_review。
7. ResearchEngine 對 admitted claims 產生 Driver / Catalyst observations。
8. 研究者或 policy 批准 revision，生成 immutable ResearchSnapshot。

7.2 Scenario-to-Portfolio

9. 建立一個或多個 Shock，組成 Scenario。
10. 鎖定 GraphSnapshot 與 PropagationRule versions。
11. ImpactEngine 產生 ImpactNodes 與 path attribution。
12. 透過 sensitivities 映射到公司 estimates 與 valuation。
13. ETFEngine 按 holding weight 聚合，輸出 coverage 與 missing list。
14. PortfolioEngine 進行 ETF look-through、直接持股去重與 currency normalization。
15. PublicBuilder 或 API 發布 versioned artifacts。

8. ID、版本與時間語義

8.1 ID 規範

```
<entity_type>:<namespace>-<identifier>
company:US-NVDA
security:US-NVDA-NASDAQ
technology:HBM4
etf:AXSM
shock:HBM4-SUPPLY-DOWN-20
```

- ID 不包含可變名稱、日期格式化文字或陣列位置。
- Snapshot ID 應由 canonicalized input references + engine version + policy version 雜湊生。
- Execution ID 表示一次執行；Snapshot ID 表示內容身份，兩者不得混用。

8.2 三種時間

時間	定義	例子
Event time	事實發生或適用的時間	季度財務 period、Catalyst expected window。
Publication time	來源公開時間	10-Q filed_at、新聞 published_at。
System time	AXIOM 取得或寫入時間	fetch_at、admitted_at、snapshot generated_at。

所有影響回測與 point-in-time 研究必須避免使用 event time 之後才公開的資料。

9. Repository、儲存與遷移

9.1 Repository Protocol

```
class Repository(Protocol):
    def get(self, entity_id: str): ...
    def list(self, *, filters): ...
    def add(self, model): ...
    def save_snapshot(self, snapshot): ...

class UnitOfWork(Protocol):
    canonical: CanonicalRepository
    evidence: EvidenceRepository
    research: ResearchRepository
    ...
    def commit(self): ...
    def rollback(self): ...
```

JSON adapter 仍是 v1.0 支援目標，便於 Git versioning 與可讀性；SQLite adapter 建議作為本機查詢與索引層。Domain 與 Engine 不得依賴其一。

9.2 資料目錄目標

```
data/
  canonical/ evidence/ research/ industry/ valuation/ etf/ scenario/ portfolio/
  executions/ snapshots/
  public/v1/
  schemas/v1/
  migrations/
  quarantine/
```

9.3 Migration 原則

- 每個 schema 變更需提供 source_version → target_version migration。
- Migration 必須 idempotent，並有 fixture 與 rollback / backup 說明。
- 舊 public artifacts 至少維持一個 minor release 的相容期。
- 資料遺失型 migration 必須明確標記 destructive 並要求人工確認。

10. CLI、Public Schema 與未來 API

10.1 CLI v1.0 命令面

命令	角色
axiom validate	完整 schema、referential integrity、temporal consistency 與 governance checks。
axiom ingest	取得或匯入文件，產生 ingestion execution。
axiom evidence	檢視 claims、evidence、admission queue。
axiom research	取得 thesis / drivers / catalysts / revisions。
axiom industry	查詢 graph、paths、exposures。
axiom value	執行或讀取 valuation books。
axiom etf	ETF profile、holdings、theme、valuation。
axiom impact	單一 shock impact。
axiom scenario	多 shock 與 Bull/Base/Bear 比較。
axiom portfolio	look-through exposure、valuation、scenario risk。
axiom build-public	輸出 versioned public artifacts。
axiom migrate	預覽與執行資料 migration。

10.2 輸出契約

```
{
  "schema_version": "1.0",
  "generated_at": "...",
  "execution_id": "...",
  "snapshot_id": "...",
  "data": {...},
  "methodology": {...},
  "coverage": {...},
  "warnings": [...]
}
```

所有 CLI 支援 --format json；人類可讀格式可另加 --format table。錯誤碼應區分 validation_error、not_found、incomplete_coverage、provider_error、internal_error。

11. Evidence Governance 與研究審計

11.1 Source Quality

等級	來源類型	預設策略
A	法定申報、公司正式文件、交易所資料	可直接進入高優先 admission queue。
B	Earnings call transcript、官方簡報、可信產業資料	需 entity/time validation。
C	主流新聞、專業研究摘要	需交叉驗證或降低 confidence。
D	社群、匿名來源、未驗證二手內容	預設不影響發布分數，僅作候選線索。

11.2 人工覆核邊界

- 高 materiality 且低 confidence 的 Driver 變更。
- Catalyst 日期移動超過政策門檻。
- 單一來源造成大幅估值或 ETF Impact 變化。
- Entity merge、relation conflict、重大 restatement。

11.3 審計事件

```
AuditEvent {
  audit_id, occurred_at, actor_type, actor_id, action,
  object_type, object_id, before_hash, after_hash,
  execution_id, rationale, policy_version
}
```

12. Validation、測試與可觀測性

12.1 Validation Layers

層級	檢查
Schema	型別、enum、range、required fields。
Referential	所有 ID 存在；禁止 orphan links。
Temporal	$\text{valid_from} \leq \text{valid_to}$ ；publication time 與 event time 合理。
Graph	禁止不允許的 cycle；edge direction / type 合法。
Financial	權重總和、單位、currency、period consistency。
Evidence	claim 至少一個 evidence link；admitted candidate 有 decision。
Snapshot	input hashes、engine/policy version、deterministic ID。
Public	schema_version、no private fields、coverage / warnings 完整。

12.2 Test Pyramid

- Model invariant tests：每個 Pydantic model 的正常、邊界與錯誤案例。
- Engine unit tests：固定 fixtures 驗證 deterministic results。
- Contract tests：JSON adapter、SQLite adapter、provider adapters。
- Golden tests：CLI JSON 與 public artifact 的穩定輸出。
- Migration tests：每個舊版本 fixture 可升級且重複執行不變。
- Release smoke：validate、research、industry、value、etf、impact、scenario、portfolio、build-public。

12.3 可觀測性

每個 application command 應產生 ExecutionRecord，記錄 start/end、status、input refs、output refs、warnings、error class、engine version、policy versions。敏感來源內容不得直接寫入 log。

13. 安全、授權與資料保護

- 來源內容保存需遵守授權；無權保存全文時，只保存 metadata、hash、locator 與允許的摘要。
- API key 與 provider token 不得進入 repository、Snapshot 或 public JSON。
- PublicBuilder 使用 allowlist schema，禁止直接序列化內部 model。
- 人工研究備註可標記 private；private 欄位不可被 public build 發布。

- 所有外部輸入視為不可信，需 parser sandbox / size limit / content validation。

14. v0.8–v1.0 Roadmap

版本	主題	主要交付	Release Gate
v0.8	Evidence Engine	RawDocument/Chunk/ Claim/EvidenceLink、 extraction run、 admission workflow、 dynamic Driver/Catalyst observations	來源到研究更新完整追 溯；candidate 不可繞過 admission。
v0.9	Scenario + Portfolio Foundation	多 Shock aggregation、 Scenario comparison、 Portfolio/Position、ETF look-through	Bull/Base/Bear 可比 較；portfolio coverage 與 double-counting policy 通過。
v0.10	Architecture Hardening	Repository protocols、 SQLite adapter、public schema v1、migration framework、audit events	JSON/SQLite contract tests；舊資料可 migration。
v1.0 RC	Calibration & Governance	Sensitivity records、 policy versioning、 golden fixtures、完整 docs	移除 production hard- coded demo coefficients；所有 release gates PASS。
v1.0	Stable Research OS	版本化 contracts、正式 CLI、public artifacts、 architecture ADRs	Semantic versioning 與 compatibility policy 生 效。

版本編號可依實際節奏調整，但能力依賴順序不應顛倒：Evidence → Scenario/Portfolio → Hardening → Calibration → Stable v1.0。

15. v1.0 驗收清單

- ☒ published Driver / Catalyst 可列出 supporting 與 contradicting claims。
- ☐ 每個 Claim 可定位到 Source / DocumentChunk，保留 extraction 與 admission metadata。
- ☐ 所有 propagating IndustryEdge 符合 cause → effect，且 graph validation 無非法 cycle。
- ☐ Impact 結果不再依賴未命名硬編碼係數；必須引用具名 policy / sensitivity。
- ☐ Bull/Base/Bear 或自訂多 Shock Scenario 可重現、可比較。
- ☐ ETF 與 Portfolio 結果明確揭露 coverage、missing data、contributors 與 confidence。
- ☐ JSON 與 SQLite adapters 通過相同 repository contract tests。
- ☐ Public artifacts 具有 schema_version，且不洩漏 internal/private fields。

- ☐ 從 v0.7 fixture 可經 migration 升級到 v1.0。
- ☐ 完整 verify_release.sh 在乾淨環境通過 ruff、pytest、CLI smoke、migration、public build。
- ☐ Architecture Decision Records 與維運文件完成。

16. Architecture Decision Records (初始集合)

ADR	決策	狀態
ADR-001	採 Modular Monolith，而非 v1.0 前拆成微服務。	Accepted
ADR-002	Industry propagation edge 固定 Cause → Effect。	Accepted
ADR-003	Published results 使用 immutable Snapshot；Execution 與 Snapshot 分離。	Accepted
ADR-004	LLM extraction 僅產生 candidate，不可直接改寫 published research。	Proposed
ADR-005	Domain/Engine 透過 Repository / Provider ports 隔離 storage 與外部服務。	Proposed
ADR-006	Public JSON 採獨立 versioned schema 與 allowlist builder。	Proposed
ADR-007	Multi-path impact 必須由顯式 aggregation policy 控制。	Proposed
ADR-008	JSON 為可讀 source-of-truth；SQLite 為支援的 indexed adapter。	Proposed

17. 主要風險與緩解

風險	可能結果	緩解
Evidence extraction hallucination	錯誤 Claim 影響研究與估值	candidate/admission 分離、來源定位、人工覆核門檻。
Graph double counting	多路徑重複放大 Impact	dependency-aware aggregation、path attribution、golden fixtures。

Coverage 被誤解	ETF/Portfolio 結果看似精確但資料不完整	coverage 置頂、missing contributors、最低 coverage policy。
Schema drift	CLI/public/adapter 不一致	versioned schema、contract tests、migration framework。
過早服務化	複雜度高於價值	維持 modular monolith，僅在明確 scaling/ownership 壓力下分。
模型係數缺乏校準	Impact 被當成預測	Demo/Production policy 分離、backtest、confidence 與 methodology。

18. 建議實作順序（下一個 30-60 天）

16. 建立 docs/architecture-v1.md 與 docs/adr/，將本文件內容轉為 repository 內可版本控制文件。
17. 新增 schema_version 與 ExecutionRecord 基礎欄位，但維持 v0.7 CLI 相容。
18. 定義 ports/repositories.py 的 Protocol，先由現有 JSON repository adapter 實作。
19. 完成 Evidence Domain 的最小模型：RawDocument、DocumentChunk、Claim、EvidenceLink、AdmissionDecision。
20. 建立 candidate → admitted 的 validator 與 CLI inspection，不先做全自動網路 ingestion。
21. 將 ResearchDriver / Catalyst 擴充為 observation + snapshot 模式。
22. 將 v0.7 hard-coded financial mapping 包裝為 DemoModelPolicy，為 Sensitivity 接口預留位置。
23. 新增 ScenarioEngine 與 multi-shock aggregation policy，再進入 Portfolio Foundation。

附錄 A：建議 Public Artifact 結構

```

data/public/v1/
  manifest.json
  search.json
  entities/{entity_id}.json
  research/{company_id}.json
  etfs/{etf_id}.json
  scenarios/{scenario_id}.json
  portfolios/{portfolio_id}.json
  schemas/*.json

```

附錄 B：命名與狀態列舉建議

類別	建議值
Admission status	candidate, admitted, rejected, needs_review, superseded

Driver trend	rising, stable, falling, uncertain
Catalyst status	identified, expected, delayed, occurred, cancelled, invalidated
Snapshot status	draft, reviewed, published, superseded
Execution status	queued, running, succeeded, partial, failed
Shock direction	positive, negative, mixed
Aggregation policy	strongest_path, independent_sum, correlated_sum, bottleneck

附錄 C : Definition of Done — 每個新功能

- Domain model 與 invariants 已定義。
- Repository contract 與至少一個 adapter 已完成。
- Application command/query 與 CLI contract 已完成。
- Unit、contract、integration、golden tests 已完成。
- Validation 與 migration 已完成。
- Public schema 影響已評估。
- Methodology、coverage、warnings 與 audit metadata 已輸出。
- README / architecture / ADR / changelog 已更新。
- verify_release.sh 在 clean venv 通過。

結語

AXIOM v1.0 的關鍵不是模型數量，而是每一個結論都能被解釋、重現與治理。當 Evidence、Research、Impact、Valuation、ETF 與 Portfolio 共享一致的 ID、時間、版本與 Snapshot 語義，AXIOM 才真正成為可長期演進的 Investment Research Operating System。